



SE2030 – Software Engineering
2026-Y2-S1-MLB-B13G2-05
Web-based Vehicle Rental System
2026.10.05
Final Report

STUDENT ID	NAME
IT25102266	Sakalasooriya S.A.A.A.
IT25102264	De Silva D.L.K.C.
IT24101349	Samaranayake P.I.S.
IT25102265	Kavindi P.D.N.
IT24101008	Wickramasinghe R.D.W.K.G.S.
IT24102871	Pahasara Y.A.P.

Table of Contents

Introduction	3
Requirements	5
Design	8
Implementation	13
Project Management	17
Conclusion & Future Work	19
Individual Contribution, Teamwork & Lessons Learned	21
References	23
Appendix	24

Introduction

Project Overview

This project presents Lanka Ride, a web-based vehicle rental system for a company that hires cars from branches in Sri Lanka. The application is built with Spring Boot on Java 17. It replaces separate paper records for vehicles, bookings, payments, and workshop jobs with one system that customers and staff share.

A guest can search by branch and travel dates and open a vehicle. A customer can request a booking, pay the rental and the deposit, and raise a support ticket. Staff approve or deny requests, record pickup and return, hold a car for maintenance, take payment, and read a daily revenue report. An administrator adds branches and staff accounts. Pages are server-rendered with JSP, Bootstrap 5, and Spring Security tags. MySQL 8 stores the records. Each person sees only the functions of their role.

Objectives

The main objective is a single, role-based system for day-to-day vehicle rental work.

The specific objectives are:

- To keep an accurate register of vehicles, branches, daily rates, and deposits.
- To show a customer only the vehicles that are free for the dates they choose.
- To carry a booking from request, through approval, payment, pickup, and return.
- To stop a vehicle that is under maintenance from being hired.
- To record rental payments, deposits, refunds, late fees, and damage charges.
- To give finance and operations a daily, monthly, and branch view of money collected.
- To let a customer open a support ticket and to keep a notification log.
- To protect accounts with hashed passwords and role checks on every staff action.

Target Users or Stakeholders

- **Customer.** Registers, searches vehicles, requests a booking, pays, views the invoice, and opens a support ticket. After login the customer lands on My trips.
- **Booking supervisor.** Approves or denies a request with a reason, then records pickup and return mileage and fuel.
- **Fleet coordinator.** Adds and updates vehicles, records branch transfers, and opens or closes maintenance.
- **Finance manager.** Records payments and refunds, opens the invoice, and reads revenue reports.
- **Operations manager.** Monitors the live booking list and reads the same operational reports.
- **Administrator.** Adds branches and creates staff accounts. Public registration cannot create this role.

Scope and Limitations

Scope:

- Customer registration, sign-in, sign-out, and password reset.
- Role-based access for the customer and the five staff roles.
- Vehicle catalogue with branch, price, deposit, features, and status.
- Date search that hides cars already booked or under maintenance.
- Booking request, approval, denial, cancellation, pickup, and return.
- Maintenance records that change vehicle availability.
- Mock payment of the rental and the deposit, with an invoice.
- Daily, monthly, and branch reports, with print and CSV export.
- Support tickets, a notification log, and administration of users and branches.

Limitations:

- There is no native mobile application.
- Payment uses a mock gateway, so no real card is charged or stored.
- Live GPS tracking of a vehicle is outside the current scope.
- Notification delivery is stored in the application log. A live mail server is not required.
- The system is aimed at a desktop browser.
- Reports show only the payment rows that staff have recorded.
- The Spring Boot process and MySQL must both be running.

Requirements

Functional Requirements

Vehicle management. The fleet coordinator registers a vehicle with its registration number, category, brand, model, seats, gearbox, fuel type, features, photo, daily rate, deposit, home branch, and status. Records can be updated, and a car can be retired. A branch transfer is stored as its own row, with the vehicle and the branches it left and joined. Guests can filter the catalogue by branch, dates, category, gearbox, fuel, and price. When dates are supplied, a car with an overlapping active booking, or a car that is not available, is left out of the results.

Booking management. A signed-in customer chooses an available vehicle, a pickup branch, and the pickup and return dates. The service checks availability again when the request is saved. The booking supervisor approves or denies the request and records a reason. Pickup stores mileage and fuel. Return stores the same pair and raises a discrepancy note when they do not match what was expected. A customer can cancel a request that has not started. Status values are pending, approved, denied, ongoing, completed, and cancelled. Payment status is stored separately, so a trip can be approved and still unpaid.

Fleet maintenance. The fleet coordinator opens a maintenance record with a type, dates, estimated and final cost, the mechanics assigned, and a description. Opening the job sets the vehicle to maintenance, which removes it from customer search. The booking service also refuses dates that overlap an open job. Closing the job returns the vehicle to available when no other job is open. Status values are open, in progress, and closed.

Payment and billing. The finance manager records a rental payment and a refundable deposit against a booking. Each movement is its own row: type, amount, success or failure, a mock gateway reference, a note, and the staff username that recorded it. A refund, a late fee, or a damage charge is another row on the same booking, so the history is not overwritten. An invoice page can be printed. Card numbers are not collected. Payment status on the booking is unpaid, pending payment, paid, or refunded.

Dashboard and reporting. The operations dashboard shows how many vehicles exist, how many are available, how many are in maintenance, how many bookings are waiting, and how many branches are open. The report page totals successful payments for a chosen day, month, or year, compares branches, and lists vehicles by location. The same page can be printed or exported as CSV. Finance and operations both use these figures. The totals are read from the payment and vehicle tables, not from a second copy of the money.

Support and notifications. A customer opens a ticket with a subject and a message. Staff can record a response and move the ticket through open, in progress, resolved, and closed. The notification log stores the channel, the recipient, and whether delivery succeeded, so a failed alert can be retried from the alerts screen even when email is not configured. Password reset uses a separate token row with an expiry.

Non-Functional Requirements

- **Security.** Passwords are stored with BCrypt. URL rules and method security both apply, so hiding a menu link is not the only control. A customer who opens a staff address receives HTTP 403.
- **Usability.** Customers land on My trips. Staff land on the operations dashboard. Both shells use the same header, colours, and card layout.
- **Reliability.** Availability is checked on the search page and again when a booking is saved, so two customers cannot take the same car for the same days.
- **Maintainability.** Vehicle, booking, payment, fleet, reports, and support each have their own controller, service, and repository.
- **Portability.** The site runs in current desktop browsers, including Chrome, Edge, and Firefox.
- **Data integrity.** Required fields are validated, and status is stored as a fixed set of values rather than free text.

Constraints or Assumptions

Technology stack constraints:

- Java 17 and Spring Boot 3.3 for the application.
- Spring MVC with JSP and Bootstrap 5 for the pages. The project does not use a separate JavaScript front end.
- Spring Data JPA and Hibernate 6 for persistence.
- MySQL 8 as the database. An H2 profile exists for tests and for a machine without MySQL.
- Maven Wrapper for the same build on every member's computer.

Deployment environment:

- The application listens on port 8080. MySQL listens on port 3306. The database name is lanka_ride, and it is created if it is missing.
- A user needs a desktop browser. No separate client install is required.

External services:

- The payment step is a mock gateway inside the application. A bank provider is not integrated.
- Email delivery is recorded in the notification log. An external mail service is not required for the demonstration.

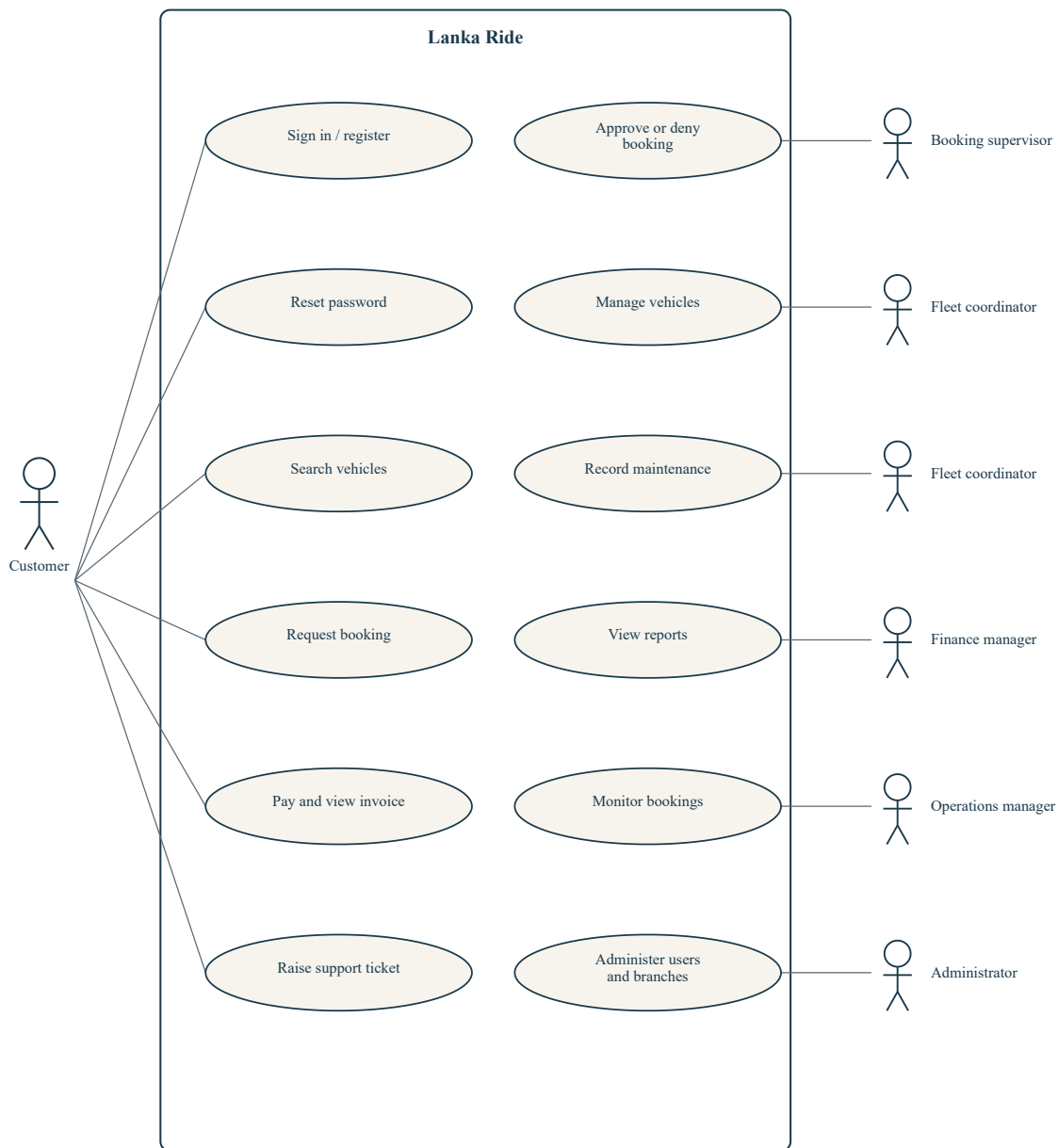
Users and devices:

- The interface is laid out for a laptop or desktop screen.
- Each account has one or more roles. Seeded demonstration accounts, listed in the repository README, are used for the viva. Public registration always creates a customer.

Ethics. Staff can see bookings they do not personally own, and an administrator can create further staff. The control is least privilege by role, hashed passwords, and a register that cannot mint an administrator. Because the gateway is a mock, the invoice must not be treated as a real card charge. A later production release would send card data only to a certified provider.

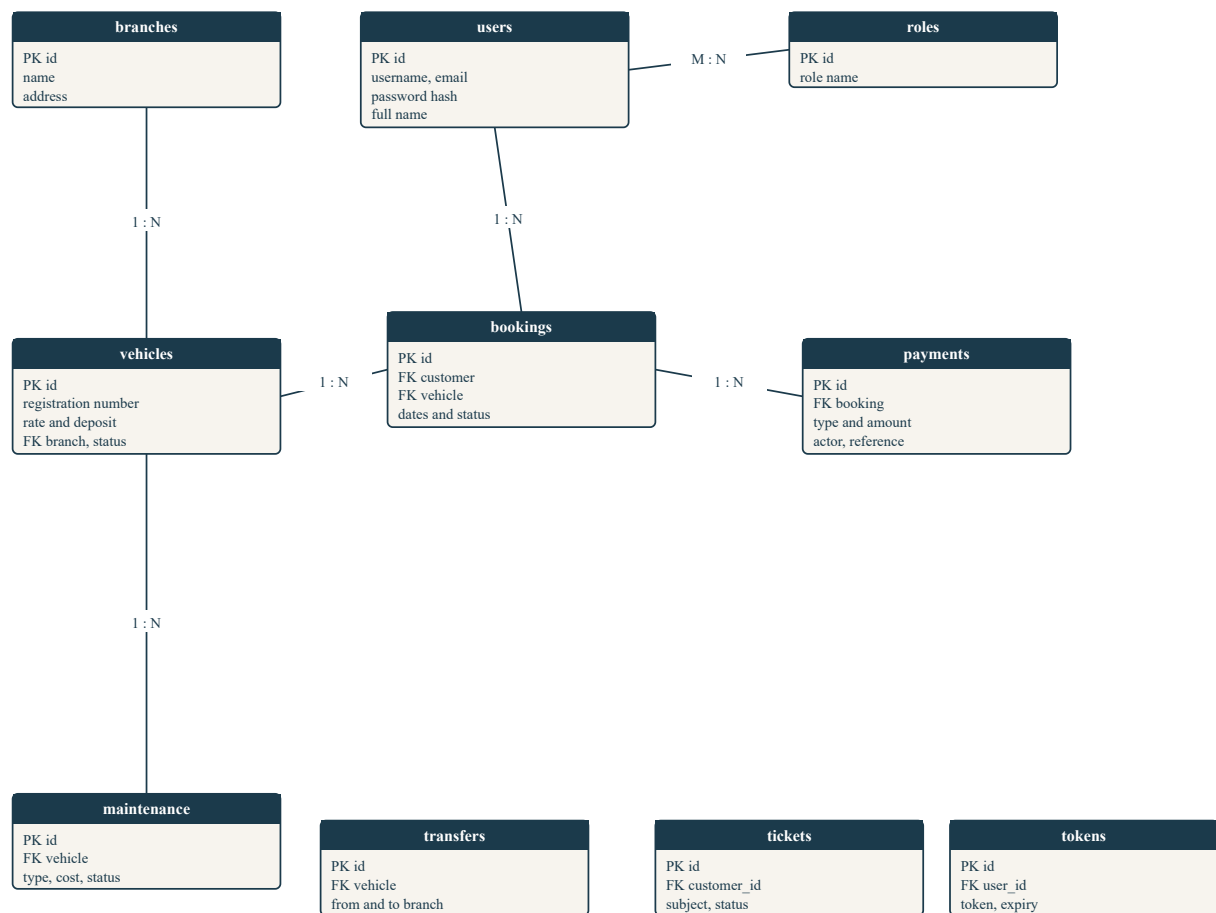
Design

Use Case Diagram



The customer owns search, booking, payment, and support. Staff use cases sit on the right. Sign-in is shared. Finance also records the payment the customer starts, and the operations manager can also approve a booking.

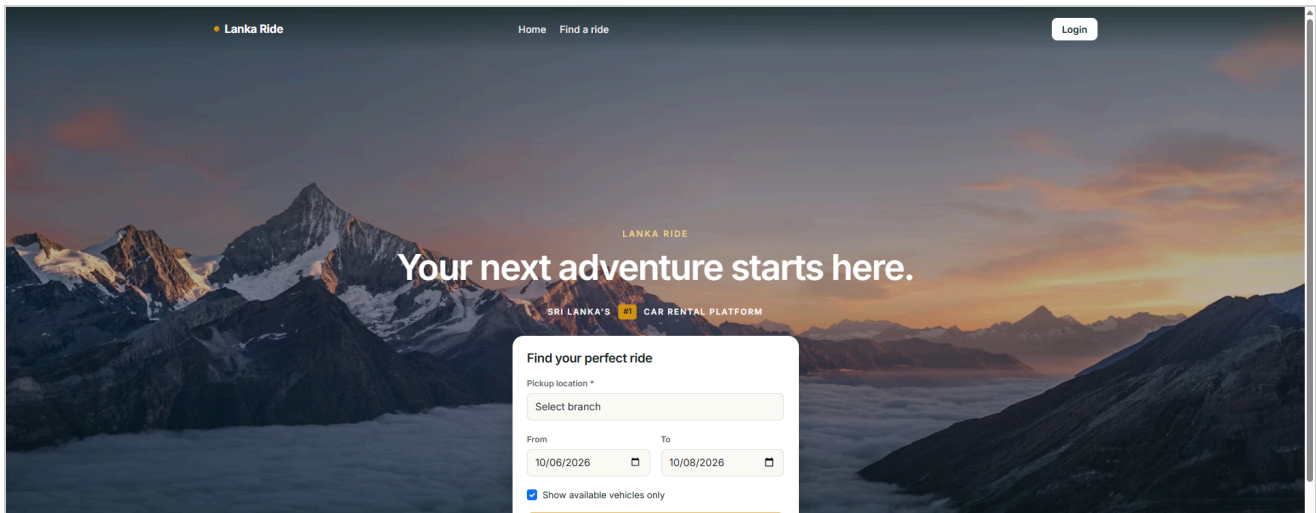
Database Design / ER Diagram



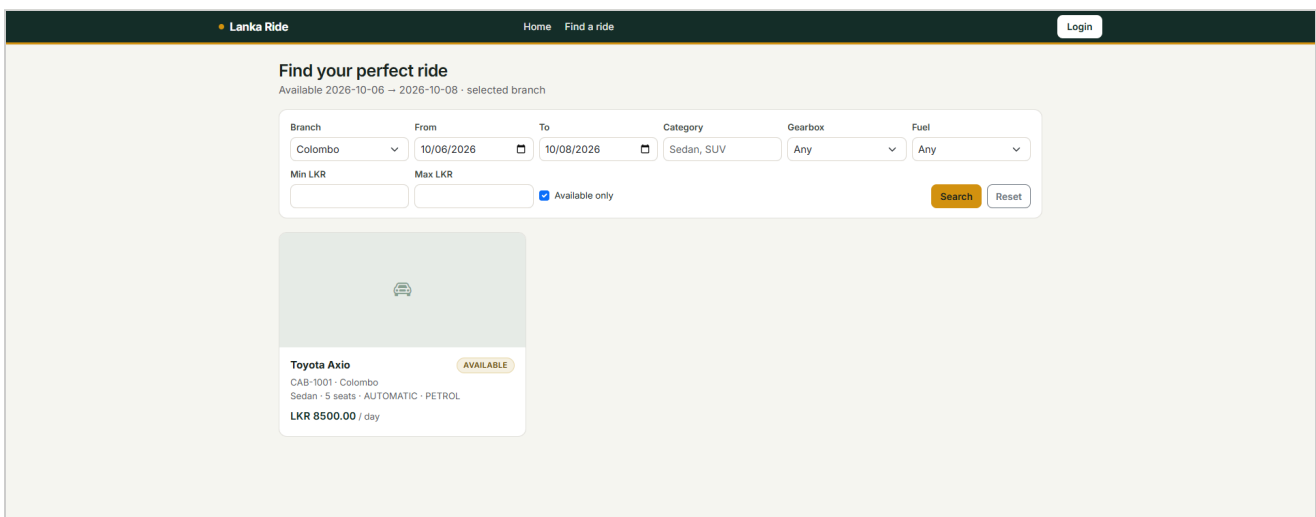
Tickets and password-reset tokens belong to a user. Branch transfers belong to a vehicle. user_roles joins users and roles.

A user has many roles through user_roles, many bookings, and many tickets. A branch has many vehicles. A vehicle has many bookings, maintenance rows, and transfers. A booking has many payment rows. PK is the primary key and FK is a foreign key.

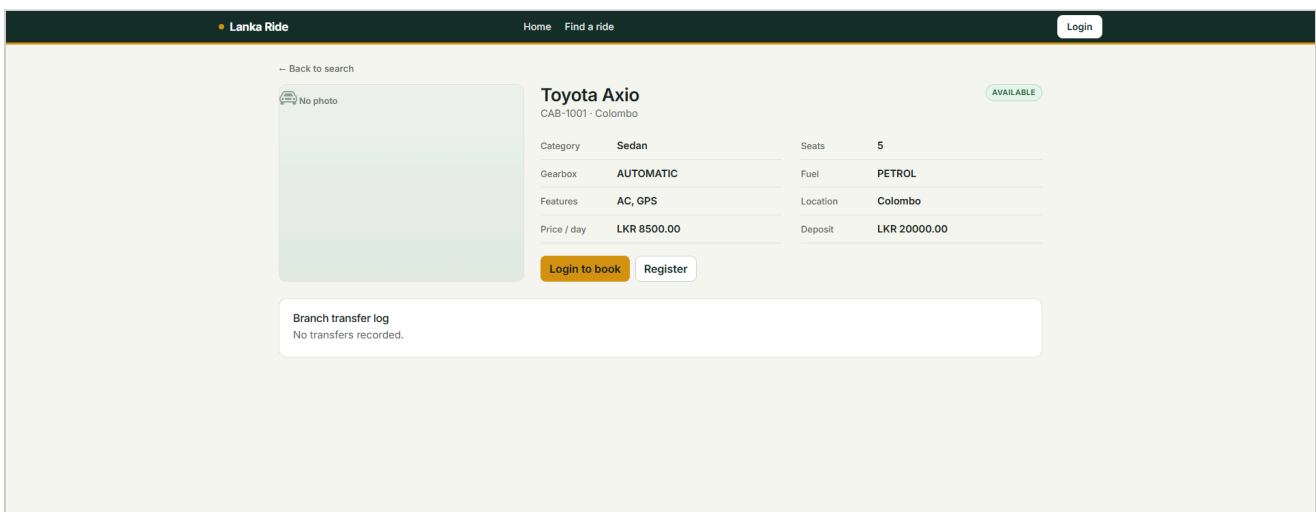
UI Sketches/Screenshots and Descriptions



Home page. The guest chooses a branch and the pickup and return dates.



Search results for Colombo, 6–8 October 2026. The Toyota Axio at Colombo is offered at the daily rate.



Vehicle page. Status, features, daily rate, and deposit are shown before a request can be made.

Lanka Ride

Create account

Public registration creates a **customer** account only.

Full name

Username

Email

Password

Confirm password

Register

Already registered? [Sign in](#)

Registration. A public account is always a customer.

Lanka Ride

Sign in

Customers go to My trips. Staff open the operations dashboard.

Username

Password

Login

Customer demo: `customer` / `customer123`

► Staff demo logins

No account? [Create one](#) - [Forgot password](#)

Sign-in. Customers continue to My trips. Staff continue to the operations dashboard.

Lanka Ride

HomeFind a rideMy tripsBookingsSupport

customerLogout

My trips

Welcome back, customer. Your trips hub — track bookings, pay invoices, and get support.

Find a ride

Active / pending1

Awaiting payment0

Help

All bookingsSupport

Recent trips

View all

BOOKING #3 - KANDY

Honda CR-V

2026-10-05 – 2026-10-05 CAB-2002

ONGOINGPAID

DetailsInvoice

Customer home. My trips shows the active booking, payment state, and the invoice link.

Main Interfaces

Home. Full-page search by branch and dates, then a card list of vehicles that are free for that window.

Register and login. Registration collects name, username, email, and password, and creates a customer only. Login sends each role to its own landing page. Password reset is a separate form.

My trips. The customer sees active and unpaid bookings, and can open details, the invoice, all bookings, or support.

Operations dashboard. Staff see fleet size, availability, maintenance count, pending bookings, and branches.

Bookings. The list shows status and payment status. A supervisor approves, denies, or records pickup and return from this area.

Payments. The ledger lists each rental, deposit, refund, or charge, with the staff actor and a link to the invoice.

Maintenance. Open and closed jobs sit with the vehicle. A new job takes the car out of search.

Reports. Daily, monthly, and branch figures, with CSV and print.

Support. Ticket list and the form a customer uses to ask for help.

Admin. Add a branch, list current branches, and create a staff user with one role.

Implementation

Tools and Technologies Used

- **Java 17.** Language and runtime for the application and the business rules.
- **Spring Boot 3.3.5.** Packages the web application, starts the embedded server, and loads configuration.
- **Spring MVC, JSP, and JSTL.** Server-rendered pages under WEB-INF/views.
- **Bootstrap 5.3.** Shared layout, forms, and cards for the customer shell and the staff shell.
- **Spring Security 6.3.** Form login, role checks, method security, and BCrypt password hashing.
- **Spring Data JPA and Hibernate 6.** Repositories and the mapping onto MySQL. The schema updates on startup.
- **MySQL 8.** Stores users, roles, branches, vehicles, bookings, payments, maintenance, transfers, tickets, notifications, and reset tokens.
- **Maven Wrapper 3.9.** Dependency management and the build, including the compiler flag that keeps request-parameter names.
- **JUnit with H2.** Completion checks that do not need the MySQL service.
- **IntelliJ IDEA.** The IDE used to build and run the application on JDK 17.
- **Git and GitHub.** One long-lived feature branch per module, integrated on the release line.

Key Features Developed

User registration and authentication. Password reset. Role-based menus and server-side checks. Vehicle catalogue and date search. Branch transfer log. Booking request, approval, denial, cancellation, pickup, and return. Maintenance that blocks hire. Mock payment, invoice, refund, late fee, and damage charge. Operations dashboard. Daily, monthly, and branch reports with CSV and print. Support tickets and notification log. Administrator creation of branches and staff users.

Screenshots of Core Functions

The screenshot shows the 'Operations dashboard' for an administrator. The top navigation bar includes 'Lanka Ride' and various menu items like Home, Find a ride, Dashboard, Bookings, Monitor, Payments, Reports, Maintenance, Add vehicle, Admin, Alerts, Support, and user actions like 'admin' and 'Logout'. The dashboard itself has a header with 'Operations dashboard' and a welcome message. Below this are five summary cards: 'Total vehicles' (4), 'Available' (2), 'In maintenance' (0), 'Pending bookings' (0), and 'Branches' (5). The main section is titled 'Your workspace' and contains several functional blocks: 'BOOKINGS Pending approvals' (0 waiting for a decision), 'FLEET Add vehicle' (Create a new fleet record), 'FLEET Maintenance' (Service records and availability), 'FINANCE Payments ledger' (Transactions, refunds, invoices), 'REPORTS Branch & period reports' (Daily / monthly / annual + utilization), 'SUPPORT Support tickets' (Respond to customer issues), and 'ADMIN System admin' (Add branches and staff users). There are also buttons for 'Browse fleet' and 'Open reports'.

Operations dashboard for an administrator, who also holds the fleet-coordinator role. Counts come from the live database.

The screenshot shows the 'Bookings' page. The top navigation bar is identical to the dashboard. The page header includes 'Bookings' and a sub-header 'Review reservation requests and open approval screens.' Below this are tabs for 'Operations monitor', 'Pending', 'All', and 'New booking'. The main content area displays a list of three bookings. Each booking card shows a number, location, vehicle type, dates, and status. The first booking is '#3 - KANDY Honda CR-V' with dates '2026-10-05 - 2026-10-05', status 'ONGOING', and 'PAID'. The second is '#2 - GALLE Toyota Axio' with dates '2026-09-18 - 2026-09-20', status 'APPROVED', and 'PENDING_PAYMENT'. The third is '#1 - KANDY Nissan Caravan' with dates '2026-09-18 - 2026-09-20', status 'CANCELLED', and 'UNPAID'. Each card has an 'Open' button.

Booking list. One trip is ongoing and paid, one is approved and awaiting payment, and one is cancelled.

Lanka Ride

[Home](#)[Find a ride](#)[Dashboard](#)[Bookings](#)[Monitor](#)[Payments](#)[Reports](#)[Maintenance](#)[Add vehicle](#)[Admin](#)[Alerts](#)[Support](#)

admin

Logout

Payment audit trail

Every sandbox charge, decline, fee, and refund, with the account that recorded it.

RENTAL - BOOKING #3

LKR 12000.00

MOCK-F2FF4C79-R 2026-10-05T15:48:47.361692 customer Rental fee

SUCCESS

Open invoice

DEPOSIT - BOOKING #3

LKR 30000.00

MOCK-F2FF4C79-D 2026-10-05T15:48:47.357678 customer Booking deposit

SUCCESS

Open invoice

Payment ledger for the Honda CR-V booking: rental LKR 12,000 and deposit LKR 30,000, each marked successful, with the invoice link.

Lanka Ride

[Home](#)[Find a ride](#)[Dashboard](#)[Bookings](#)[Monitor](#)[Payments](#)[Reports](#)[Maintenance](#)[Add vehicle](#)[Admin](#)[Alerts](#)[Support](#)

admin

Logout

Fleet maintenance

Service and repair records that affect vehicle availability.

Send due reminders

Add record

#1 - REPAIR

Honda CR-V

CAB-2002 2026-10-02 LKR 500000.00

CLOSED

Open

#2 - DAMAGE

Honda CR-V

CAB-2002 2026-09-21 LKR 50000.00

CLOSED

Open

Closed repair and damage records on the same vehicle. An open job would set the vehicle to maintenance.

Reports
Daily — 2026-10-05

Download CSV Print

Period: Daily Reference date: 10/05/2026 Branch filter: All branches Apply filters

Collected LKR 42000.00	Refunds LKR 0.00	Net LKR 42000.00	Bookings / completed 1 / 0
----------------------------------	----------------------------	----------------------------	--------------------------------------

Branch comparison

BRANCH	Revenue LKR	Fleet	Bookings	UTIL
Colombo	0.00	1	0	0.0% UTIL
Kandy	42000.00	0	1	0% UTIL
Galle	0.00	1	0	0.0% UTIL

Daily report for 5 October 2026. Collected and net amounts match the two successful payment rows. CSV and print are on the page.

Support centre
Triage customer tickets and post staff replies.

+ New ticket

TICKET #1 - DEMO CUSTOMER (@CUSTOMER)
Change pickup type for booking
2026-10-05T15:59:33.856116 personal reason

OPEN Respond

Support centre. Ticket 1 is open, raised by the demonstration customer.

System admin
Scale the platform — add branches and staff accounts. Public register remains customer-only.

Add branch
New depot locations appear in search and booking forms.

Name
e.g. Negombo

Address
Street / area

Add branch

Current branches

- Colombo**
Colombo Fort Depot
- Kandy**
Kandy City Branch
- Galle**
Galle Face Branch

Add user
Create staff roles — never use public register for admin accounts.

Full name

Username

Email

Password

Role
CUSTOMER

Create user

7 users in the system

Administration. Current branches are Colombo, Kandy, Galle, Kaduwela, and Matara. New staff are created here, not on the public register.

Project Management

Agile Approach and Sprint Summary

Lanka Ride was built with Scrum. Samaranayake was Scrum Master and Wickramasinghe was Product Owner. The other four members were developers. Each member owned one use case and one long-lived feature branch. Work was integrated onto the shared release line once that module could be shown against the common database.

The order of sprints followed the data dependencies. A payment cannot exist before a booking, and a revenue report cannot exist before payment rows. Authentication was shared in the first sprint because every later screen needs it. GitHub holds the repository, the feature branches, and the integrated release.

Sprint	Main Activities
Sprint 1	Project setup, MySQL, registration, sign-in, password hashing, role-based landing pages, and the vehicle catalogue with search.
Sprint 2	Booking request, approval and denial, cancellation, and maintenance that removes a vehicle from hire.
Sprint 3	Rental and deposit payments, invoice, refunds, late and damage charges, and the pickup and return checklist.
Sprint 4	Operations dashboard, daily and branch reports, CSV and print, support tickets, notification log, and administration of branches and staff.

The main integration risk was the shared navigation bar and the security rules, because every module adds a menu entry. Those files were treated as integration points and reviewed when a feature branch was brought across, rather than letting each branch invent its own login.

Task Distribution Among Team Members

Member	Main Contribution
Sakalasooriya S.A.A.A.	Vehicle management, catalogue search, and system administration. Shared the first-sprint authentication work.
De Silva D.L.K.C.	Fleet maintenance, and the rule that a vehicle under service cannot be booked.
Samaranayake P.I.S.	Scrum Master. Booking and reservation, including approval and the return mileage check.
Kavindi P.D.N.	Payment and billing: ledger rows, invoice, refunds, and late or damage charges.
Wickramasinghe R.D.W.K.G.S.	Product Owner. Operations dashboard and revenue, branch, and location reports.
Pahasara Y.A.P.	Customer support, notification log, and a pass over role menus. Shared the first-sprint authentication work.

Development Phases or Timeline

Phase	Period	Key Milestones / Deliverables
Project initialisation and planning	Start of the module	Group agreement, backlog, stack choice, and the repository.
Sprint 1: accounts and vehicles	About week 6–7	Authentication, roles, vehicle records, and basic search. About 25 percent of the product.
Sprint 2: booking and maintenance	About week 8–9	Request and approval, and a maintenance hold that blocks hire. About 50 percent.
Sprint 3: payment and return	About week 10–11	Payments, invoice, refunds, and return mileage. Dashboards started. About 75 percent.
Sprint 4: reports and support	About week 12–14	Reports, tickets, notification log, integration, and the final report. 100 percent of the six use cases.
Submission	5 October 2026	Final report, demonstration build, and evaluation.

Conclusion & Future Work

Summary of Achievements

Lanka Ride runs as one Spring Boot application on MySQL. The six agreed areas are present: vehicles, maintenance, booking, payment, reporting, and support, together with sign-in, password reset, and administration.

A guest can search a branch for a date range. A customer can request a trip, see it under My trips, and open an invoice after payment. A supervisor can approve or deny. Finance can record the rental and the deposit. Fleet can hold a car for maintenance so it leaves the search results. Operations can read a daily report and export it. An administrator can add a branch and a staff user. The screens in this report were captured from that running system on 5 October 2026.

Challenges Faced

The hardest design choice was keeping three statuses apart: whether the vehicle can be hired, how far the booking has progressed, and whether the money has been taken. One combined field would have hidden an approved booking that is still unpaid, or a paid car that later goes to the workshop. Splitting the fields added checks in the service layer and removed false “available” results.

A second difficulty was access control. Hiding a link does not stop a customer from typing a staff address. URL rules and method security were both required. A third difficulty was integration. Modules were developed on separate branches, and the shared layout and security configuration produced conflicts when those branches were combined. The group resolved them by reviewing those shared files at integration time and by testing a full journey: search, request, approve, pay, and report.

Suggestions for Improvement or Extension

The system could be extended later by adding:

- A certified payment provider in place of the mock gateway, with card data kept at the provider.
- Live email delivery for tickets, password reset, and maintenance reminders.
- A layout pass so My trips is comfortable on a phone.
- A fuller audit of who changed a vehicle's status.
- Scheduled backups of the MySQL database.
- Deployment beyond a single local server, so a branch desk can reach the same data.
- More automated tests around overlapping dates and role denial.

None of these is required to demonstrate the six use cases. The boundary for any later payment work stays the same: card details must not be stored in Lanka Ride, and a staff account must keep only the role that needs it.

Individual Contribution, Teamwork & Lessons Learned

Name	Specific Role	Challenges Faced	How Challenges Were Overcome	Key Lessons Learned	Reflection on the Project
Sakalasooriya S.A.A.A. IT25102266	Developer for vehicle management and system administration (UC-02). Shared sprint-1 authentication.	Search had to ignore cars already promised or in the workshop, and the same account model had to serve customers and staff.	Availability is applied in the vehicle query and again when a booking is saved. Staff accounts are created only by an administrator.	A catalogue is a set of rules about time and status, not only a list of rows.	Owning the first sprint meant later members could sign in and attach bookings to real vehicles. I would lock the security rules before each new menu item is added.
De Silva D.L.K.C. IT25102264	Developer for fleet maintenance (UC-01).	A car in service could still appear in search if maintenance and booking did not share one status.	Opening a job sets the vehicle to maintenance. The booking check refuses dates that overlap an open job. Closing the job restores availability.	A rule that crosses modules belongs in the service that writes the booking, not only on the maintenance screen.	The useful test was trying to book a car I had just marked as under repair. When that request was refused, the two modules were actually integrated.
Samaranayake P.I.S. IT24101349	Scrum Master and developer for booking and reservation (UC-03), including return mileage.	A booking has more states than yes or no, and return mileage must flag a discrepancy without wiping the payment history.	Booking status, payment status, and the discrepancy note are separate fields. The sprint order put booking before payment so the foreign key existed first.	Scrum works when the sprint goal is something the next person can call.	Keeping the sequence honest mattered more than adding screens early. I would keep a written definition of done for “approved but unpaid” so a demo cannot skip it.

Name	Specific Role	Challenges Faced	How Challenges Were Overcome	Key Lessons Learned	Reflection on the Project
Kavindi P.D.N. IT25102265	Developer for payment and billing (UC-04).	A single balance column would hide whether money was a rental, a deposit, a refund, or a damage charge, and who recorded it.	Each movement is its own payment row with type, actor, mock reference, and outcome. The invoice reads those rows. No card number is stored.	A ledger that only inserts is easier to explain, and to report on, than a balance that is edited in place.	I treated the mock gateway as a boundary. Replacing it later should not require a new booking model.
Wickramasinghe R.D.W.K.G.S. IT24101008	Product Owner and developer for the dashboard and reports (UC-05).	Managers wanted one dashboard, but revenue, fleet location, and booking counts come from three modules finished at different times.	Reports read the payment and vehicle rows rather than keeping a second set of totals. Daily, monthly, branch, and CSV views share that query.	A report is only as current as the tables it reads. It should not cache a private copy of the money.	As product owner I pushed to finish the write-paths before the charts. The daily report is the check that payment and reporting describe the same rupees.
Pahasara Y.A.P. IT24102871	Developer for support and notifications (UC-06), and for checks across role menus. Shared sprint-1 authentication.	Support had to exist even when email was not configured, and a failed alert had to stay visible.	Tickets are stored regardless of mail. The notification log keeps channel, recipient, and delivery state so a failed send can be retried.	A feature that depends on an outside service needs a local record of what was attempted.	Walking through every role's menu caught links that led to a forbidden page. I would repeat that pass at the end of each sprint, not only at the release.

References

1. Oracle, “JDK 17 Documentation,” Oracle Java Documentation. [Online]. Available: <https://docs.oracle.com/en/java/javase/17/>. [Accessed: Oct. 5, 2026].
2. VMware, “Spring Boot Reference Documentation,” version 3.3.5. [Online]. Available: <https://docs.spring.io/spring-boot/docs/3.3.5/reference/html/>. [Accessed: Oct. 5, 2026].
3. VMware, “Spring Security Reference,” version 6.3. [Online]. Available: <https://docs.spring.io/spring-security/reference/6.3/>. [Accessed: Oct. 5, 2026].
4. VMware, “Spring Data JPA Reference Documentation.” [Online]. Available: <https://docs.spring.io/spring-data/jpa/reference/>. [Accessed: Oct. 5, 2026].
5. Red Hat, “Hibernate ORM 6.5 User Guide.” [Online]. Available: https://docs.jboss.org/hibernate/orm/6.5/userguide/html_single/. [Accessed: Oct. 5, 2026].
6. Oracle, “MySQL 8.0 Reference Manual.” [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>. [Accessed: Oct. 5, 2026].
7. Bootstrap team, “Bootstrap 5.3 documentation.” [Online]. Available: <https://getbootstrap.com/docs/5.3/>. [Accessed: Oct. 5, 2026].
8. Apache Software Foundation, “Maven Getting Started Guide.” [Online]. Available: <https://maven.apache.org/guides/getting-started/>. [Accessed: Oct. 5, 2026].
9. Git, “Git User Manual.” [Online]. Available: <https://git-scm.com/docs/user-manual.html>. [Accessed: Oct. 5, 2026].
10. GitHub, “About branches.” [Online]. Available: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches>. [Accessed: Oct. 5, 2026].
11. N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proc. USENIX Annual Technical Conference*, 1999, pp. 81–91.

Appendix

GitHub Repository

Main project repository:

<https://github.com/IT25102266/lanka-ride>

The integrated line used for this report is **release/100-percent**. Feature branches follow the module owners: feat/sakalasooriya-vehicle, feat/desilva-fleet, feat/samaranayake-booking, feat/kavindi-payment, feat/wickramasinghe-dashboard, and feat/pahasara-support.

Meeting Summary

- Sprint planning set one module goal at a time, in the order authentication and vehicles, then booking and maintenance, then payment and return, then reports and support.
- The Scrum Master kept the sequence tied to dependencies. The Product Owner checked that a report described the same payments the finance module had stored.
- Each member worked on a feature branch and brought it across only after that module could be demonstrated against the shared MySQL database.
- Integration meetings were used to resolve conflicts in the shared layout and the security rules, and to walk a booking from search through payment to the daily report.
- The final check was a role pass: a customer reaches My trips, a staff user reaches the dashboard, and a customer who opens a staff URL is refused.

How to run the demonstration

1. Install JDK 17 and start MySQL 8 with the lanka_ride database.
2. From the project root, run mvn spring-boot:run.
3. Open <http://localhost:8080>. Seeded demonstration accounts and their roles are listed in the repository README.